

Visualisasi Data dengan Pandas

Sejauh ini, kita telah menggunakan `matplotlib.pyplot` secara langsung. Namun, Pandas memiliki metode `.plot()` bawaan yang merupakan *wrapper* (pembungkus) dari Matplotlib, memungkinkan Anda membuat visualisasi dengan sintaks yang lebih ringkas.

1. Histogram

Untuk melihat distribusi dari sebuah kolom numerik.

```
import matplotlib.pyplot as plt

# Membuat histogram dari kolom 'Total Pendapatan'
df_penjualan['Total Pendapatan'].plot(kind='hist')
plt.show()
```

[Gambar histogram distribusi pendapatan]

2. Bar Plot

Sangat berguna untuk membandingkan data antar kategori. Biasanya digunakan setelah `.groupby()`.

```
# Menghitung total pendapatan per kategori
pendapatan_per_kategori =
df_penjualan.groupby('Kategori')['Total Pendapatan'].sum()

# Membuat bar plot
pendapatan_per_kategori.plot(kind='bar', title='Total
Pendapatan per Kategori')
plt.show()
```

3. Line Plot

Ideal untuk melihat tren dari waktu ke waktu (*time series*).

```
# Mengatur tanggal sebagai indeks
df_harian = df_penjualan.set_index('Tanggal')

# Membuat line plot
df_harian['Total Pendapatan'].plot(kind='line')
plt.show()
```

4. Scatter Plot

Digunakan untuk melihat hubungan antara dua variabel numerik.

```
# Membuat scatter plot antara Harga dan Jumlah Terjual
df_penjualan.plot(x='Harga Satuan', y='Jumlah Terjual',
kind='scatter')
plt.show()
```

[Gambar scatter plot harga vs jumlah terjual]

Latihan Singkat

Gunakan DataFrame `df_penjualan` dari proyek kita sebelumnya.

1. Buatlah **histogram** untuk kolom 'Jumlah Terjual'.
2. Buatlah **bar plot** untuk menampilkan **rata-rata harga satuan** per 'Produk'.

Manipulasi Data: Menangani Nilai yang Hilang

Dalam dataset nyata, seringkali ada data yang tidak lengkap. Pandas merepresentasikan data yang hilang ini dengan nilai khusus yang disebut **NaN** (Not a Number). Nilai NaN ini bisa menyebabkan masalah saat melakukan perhitungan (misalnya, menghitung rata-rata akan menghasilkan error atau hasil yang tidak valid). Oleh karena itu, kita harus tahu cara mendeteksi dan menanganinya.

1. Mendeteksi Nilai yang Hilang

Ada beberapa metode berantai yang sangat berguna untuk ini:

- `.isna()`: Metode ini akan mengembalikan DataFrame dengan ukuran yang sama, tetapi isinya adalah nilai boolean (`True` jika nilainya NaN, dan `False` jika tidak).
- `.isna().any()`: Memberi tahu Anda kolom mana saja yang **memiliki setidaknya satu** nilai yang hilang.
- `.isna().sum()`: Cara yang paling umum. Ini akan menghitung **jumlah total nilai NaN** untuk setiap kolom.

Contoh:

```
# Menghitung berapa banyak nilai NaN di setiap kolom
print(df_penjualan.isna().sum())
```

[Gambar dari output `.isna().sum()` di Pandas]

2. Menangani Nilai yang Hilang

Setelah kita tahu di mana nilai yang hilang berada, ada dua strategi utama:

a. Menghapus (Dropping) Jika jumlah data yang hilang sedikit, cara termudah adalah dengan menghapus baris yang mengandung NaN.

```
# Menghapus semua baris yang memiliki setidaknya satu nilai NaN
df_penjualan.dropna()
```

b. Mengganti (Replacing/Filling) Seringkali, kita tidak ingin kehilangan seluruh baris data. Alternatifnya adalah mengganti nilai NaN dengan nilai lain.

```
# Mengganti semua nilai NaN dengan 0
df_penjualan.fillna(0)
```

Anda juga bisa menggantinya dengan nilai yang lebih representatif, seperti rata-rata dari kolom tersebut:

```
rata_rata_pendapatan = df_penjualan['Total Pendapatan'].mean()
df_penjualan['Total Pendapatan'].fillna(rata_rata_pendapatan)
```

Latihan Singkat

Mari kita buat sebuah DataFrame baru dengan beberapa nilai yang hilang.

```
import pandas as pd
import numpy as np

data_review = {
    'produk': ['Laptop', 'Mouse', 'Keyboard', 'Monitor'],
    'skor_review': [8.5, 9.0, np.nan, 8.0],
    'jumlah_ulasan': [150, 200, 120, np.nan]
}
df_review = pd.DataFrame(data_review)
```

1. Gunakan satu baris kode untuk melihat **jumlah total nilai yang hilang** di setiap kolom `df_review`.
2. Gantilah nilai yang hilang di kolom `'jumlah_ulasan'` dengan angka 0